What You Are Actually Measuring When You Benchmark Speculative Decoding Under FP8

A measurement case study on vLLM 0.29.0 and NVIDIA L4

Akshath Tiwari
Independent
tiwariakshath@gmail.com

October 10, 2026

Abstract

Speculative decoding and FP8 quantization are both standard in production LLM serving, and their interaction is almost always reported from single runs. We set out to measure that interaction on Ada-class hardware and could not, four times, until the measurement apparatus itself was fixed.

We identify and quantify four confounds on vLLM 0.29.0 / NVIDIA L4 (SM89). First, attention-backend selection is *conditioned on KV-cache dtype*, so an FP8-versus-BF16 comparison under default settings silently swaps the attention kernel as well; target and draft models select backends independently, so pinning one does not pin the other. Second, speculative throughput is *unreproducible across boots* of an identical configuration while acceptance is not — and the dispersion is itself unreproducible. Two 12-boot measurements of the same configuration on the same engine release give CV 13.92% and 3.73%. What does replicate is the ratio to an `--enforce-eager` control measured in the same container: the CUDA-graph path delivers 0.487 and 0.768 of eager throughput on 0.29.0, and 1.014 and 1.017 on 0.30.0 — two sessions on different cards agreeing to within 0.3%. The mechanism remains unidentified, and the two checkable candidates, graph mode and backend selection, are ruled out by the engine log. Third, acceptance length τ has a boot-to-boot *coefficient of variation of 0.62%* under repeated identical boots, which we could not find reported; our own FP8-precision comparison is unreplicated and we withdraw the equivalence reading we previously gave it. Fourth, the metric commonly called *inter-token latency is, when computed from stream-arrival timestamps, inter-chunk latency*: a speculative step emits every accepted token in one stream chunk, so the measured gap exceeds the amortised production time per emitted token by roughly τ.

The first three add error; the fourth reverses the ranking. Under a standard p95-ITL service objective, every speculative configuration we measured scores *zero goodput at concurrency 64 while producing the most tokens per second in the grid.* We report FP8 × speculative results with three of the four controlled — draft-side backend selection is not, because the engine offers no way to pin it — and release the harness, the raw records, and the full history of our own corrections — preserved analysis revisions, including two that withdrew headline claims of this paper, each traceable to the same error shape.

1 Introduction

Speculative decoding trades a small draft model's cheap guesses against the target model's expensive verification [1, 2]. Its benefit is governed almost entirely by one quantity: τ, the mean

number of tokens emitted per verification step (accepted draft tokens plus the bonus token). We call this *acceptance length* rather than an acceptance *rate*, since $\tau > 1$ by construction and is not a proportion. FP8 quantization independently reduces weight and KV-cache footprint, which matters because KV capacity is what binds under concurrency [3]. Serving stacks ship both. The obvious question — does quantization degrade acceptance, and what does the combination buy under load — is asked frequently and answered from single-run measurements.

We could not answer it. Four separate times, a result we were about to report turned out to be produced by something other than the variable under study. This paper reports those four things, because they are larger than the effect they obscured.

Contributions.

1. **Backend substitution under dtype change** (§3). vLLM's `auto` backend selection is conditioned on KV-cache dtype, and target and draft select independently. A precision A/B under defaults is a kernel A/B as well.
2. **Unreproducible speculative throughput** (§4). Identical configurations disperse across boots at constant acceptance, by CV 13.92% in one session and 3.73% in another: the dispersion is real in both and its magnitude is not a property of the configuration. The reproducible quantity is the ratio to a same-container eager control — 0.487 and 0.768 on 0.29.0 against 1.014 and 1.017 on 0.30.0. We do not identify the mechanism, and we report a draft-model localisation as suggestive rather than established. `--enforce-eager` removes the variance but is not a free control — it costs FP8-weight configurations a fifth to a half of their throughput and cannot boot speculative FP8-KV cells at all on this card.
3. **Acceptance dispersion** (§5). τ has CV 0.62% across 12 fixed-prompt boots. We withdraw two claims of our own that rested on one boot per arm, and decline to convert the dispersion into an equivalence bound for a comparison measured at a different operating point.
4. **ITL is inter-chunk latency** (§6). A speculative step emits every accepted token in one stream chunk, so a p95-ITL objective ranks the highest-throughput configuration in the grid as fully non-compliant. This one does not add error — it inverts the decision.
5. **Measurements after diagnosing four confounds** (§7): compatibility, KV capacity, task accuracy at n=256, and goodput. Three of the four are controlled; draft-side backend selection is not, because the engine offers no way to pin it.
6. **A fully auditable artifact** (§10): append-only raw records, script-regenerated derived tables, provenance checked by a script that fails the build, and every retraction preserved rather than removed.

We emphasise the last because it is what made the others findable. Most of our corrections were caught by re-reading stored records against a hypothesis formed later — impossible if results are summarised at write time.

2 Setup

L4 was chosen deliberately: it is the compute capability named in the upstream reports that motivated the study, and at 22 GiB the KV-capacity constraint actually binds, which it does not on an 80 GB card.

Engine	vLLM 0.29.0, pinned by image; §4 adds a 0.30.0 replication
GPU	NVIDIA L4 (SM89, Ada, 24 GB nominal / 22.03 GiB usable), rented per second
Target	Qwen3-4B
Draft	`z-lab/Qwen3-4B-DFlash-b16`, block size 16, 5 speculative tokens
Also	n-gram (prompt-lookup) speculation, no draft checkpoint
Workload	GSM8K [4], sampled without replacement
Base image	`nvidia/cuda:13.0.3-devel-ubuntu24.04`

The harness never imports the engine. It launches vLLM as a separate server process and communicates with it over HTTP. This prevents the experiment from using an in-process path unavailable to deployments, and it includes server-side request handling, scheduling, batching and streaming. It does *not* reproduce every production environment, and it does not eliminate client or network overhead.

Every measurement is content-addressed. A configuration hashes to a `cell_id`; identical configuration yields an identical id on any machine. Records are append-only, so a re-run supersedes rather than overwrites and the superseded record remains readable. Several of our corrections depend on that history being intact.

Definition. We define

$$\tau \;=\; 1 + \frac{\text{accepted draft tokens}}{\text{verification steps}}.$$

The +1 is the bonus token, which vLLM's counter excludes. Omitting it understates τ by exactly 1 and silently changes every speedup ratio derived from it.

3 Confound 1: the attention backend moves when the dtype moves

vLLM's `auto` attention-backend selection is a function of KV-cache dtype. At BF16 KV it selects FlashAttention-2; at `fp8_e4m3` it selects FlashInfer. An experimenter varying only precision therefore varies the attention kernel too, and any resulting difference is jointly attributable.

It is worse than one substitution. Target and draft models select backends *independently*, at two distinct call sites, so pinning the backend affects the target while the draft continues to select on its own. A pinned-backend precision comparison is still confounded on the draft side for speculative cells, which we record per cell rather than assert away.

Compatibility is not uniform. Of the four attention backends vLLM 0.29.0 offers on SM89, FP8 KV runs on exactly two.

Table 1: FP8 KV cache support by attention backend, Qwen3-4B + DFlash, SM89. `launch_failed` is rejection at configuration time.

Backend	FP8 KV status	KV tokens
FLASHINFER	ok	115,920
TRITON_ATTN	ok	129,584
FLASH_ATTN	launch_failed	—
FLEX_ATTENTION	launch_failed	—
`auto`	ok (selects FLASHINFER)	115,920

This is the mechanism behind the dtype-conditioned selection: `auto` does not switch away from FlashAttention-2 under FP8 KV as a preference, it switches because FlashAttention-2 *cannot* serve FP8 KV on this hardware. The original hypothesis motivating this study — that FlashInfer's FP8 attention kernels are SM90-only, so FP8 KV plus non-causal drafting would crash on Ada — is refuted. The combination runs.

Our own error here. We initially concluded that the backend selection flag was accepted, echoed back, validated against, and then ignored, and reported this upstream. That was wrong: the flag controls the target, and what we observed was the draft selecting independently. We corrected the report. The mistake is instructive because we compared requested-versus-selected backend for *a server* when there is no such thing as a server's backend — there are two.

4 Confound 2: speculative throughput is unreproducible across boots, and so is the dispersion itself

Booting an identical speculative configuration repeatedly produces widely scattered throughput while acceptance stays put. We measured this twice, six days apart, and report both because they disagree:

Engine	Card	Arm	Session	Mean tok/s	CV
0.29.0	L4	default	09-23	39.2	**13.92%**
0.29.0	L4	default	09-29 a	61.2	**3.73%**
0.30.0	L4	default	09-29 b	82.8	0.99%
0.30.0	L4	default	09-29 c	78.2	1.61%
0.29.0	**A10**	default	09-30	**132.6**	**2.08%**
0.29.0	L4	eager	09-23	80.5	1.44%
0.29.0	L4	eager	09-29 a	79.7	1.69%
0.30.0	L4	eager	09-29 b	81.6	1.28%
0.30.0	L4	eager	09-29 c	77.0	2.79%
0.29.0	**A10**	eager	09-30	**138.3**	1.28%

Each row is 12 fixed-prompt boots inside a single container (the 09-23 eager row is 8 and the 09-29 c eager row is 11, truncated by a runner timeout). Sessions 09-29 a, b and c ran on three distinct physical cards, recorded by UUID; 09-23 predates that recording.

Both 0.29.0 default rows measure the same declared configuration on the same engine release and differ by a factor of 3.7 in dispersion and 1.6 in level. **The 0.30.0 rows do not:** 0.99% and 1.61%, on different cards. The low dispersion replicates where the high dispersion did not.

The eager arm is the control, and it does not move. 80.5, 79.7 and 81.6 tok/s across three sessions, two engine releases and at least two distinct physical cards, with CV between 1.28% and 1.69%. Normalising each default measurement by the eager measurement taken beside it therefore removes session and card effects:

Engine	Card	Session	default / eager
0.29.0	L4	09-23	0.487
0.29.0	L4	09-29 a	0.768
0.30.0	L4	09-29 b	**1.014**
0.30.0	L4	09-29 c	**1.017**
0.29.0	**A10**	09-30	**0.959**

On 0.29.0 the CUDA-graph path costs between 23% and 51% of the eager path's throughput on the L4, and how much it costs does not reproduce. On 0.30.0 it reaches parity, and *that* reproduces: two sessions on different cards agree to within 0.3%, against 58% disagreement between the two 0.29.0 sessions.

The deficit is largely specific to the L4. Running the identical 0.29.0 sweep on an NVIDIA A10 (SM86, Ampere) gives a ratio of 0.959 and a dispersion of 2.08%. Same engine release, different card, and a 23–51% deficit becomes 4%. So §4 is not a general property of speculative decoding under CUDA graphs, and the **L4** in this paper's title is load-bearing rather than merely descriptive.

An earlier revision attributed this to memory capacity. That was wrong, and we had not checked it against our own records: both cards report **22.5 GiB**. Capacity is not the difference between them.

What does differ is sustained power. The L4 is a **72 W** part running at 2040 MHz; the A10 is a **150 W** part at 1695 MHz. A card with less than half the power budget can throttle under the faster execution path and not under the slower one, which is the shape of what we observe: the default arm is both slower and more variable, the eager arm is neither, and the gap closes on the card with headroom. Architecture (Ada versus Ampere) remains a competing reading.

We then measured it, and it is wrong. Sampling achieved clocks, power draw and the driver's own throttle reasons across the measurement window gives, on one L4 session at vLLM 0.29.0: the default arm power-capped for 84% of samples at 1991 MHz and 67.2 W producing 56.6 tok/s, against the eager arm capped for **95%** of samples at **1645 MHz** and **69.8 W** producing **78.1** tok/s. Thermal throttling is 0.0% throughout. The faster arm is capped more, draws more, and clocks lower. Power capping is near-saturated in both arms and therefore cannot distinguish them.

What the measurement does show is that the slow arm holds a *higher* boost clock while producing *fewer* tokens. A card clocking up and delivering less work is not at a ceiling; it is idle or stalled between bursts. That is a third hypothesis, it is cheap to test with `utilization.gpu`, and we do not claim it either. Architecture remains the other surviving reading. We report the A10 as one session; by the argument above its 2.08% is a draw, not a property.

The ratio is the quantity to report. Session 09-29 c ran 5.6% slower than 09-29 b on *both* arms (default 82.8 to 78.2, eager 81.6 to 77.0). Because both moved together, the eager arm identifies that as a property of the card rather than of the execution path, and the ratio absorbs it. Without a control measured in the same container it would have read as a 5.6% replication failure.

An earlier revision of this paper reported only the first row. It gave 13.92% as *the* boot dispersion of this configuration. A second measurement puts the same quantity at 3.73%. The effect is present in both — the default arm disperses more than the eager arm beside it in every

session we have run — but the magnitude is a property of the session, not of the configuration, and quoting one number for it was wrong.

We applied that standard to our own new figures rather than only to the old one. An earlier version of this section reported the 0.30.0 result from a single session; by the argument above that is a draw, so we measured it again on another card before quoting it. A dispersion measured once is a draw, not a property — the same error as reporting a mean from one run, applied one level up — and it survived four revisions of this paper because a clean number with $n = 12$ does not look like it needs replicating.

Two measurement choices matter here, and we got both wrong in earlier revisions. First, we report CV rather than a max/min range: a range grows with sample size by construction, so ranges from arms with different boot counts cannot be compared. Second, we restrict to *fixed-prompt* boots. Pooling in grid records, whose repeats draw different prompts by design, mixes prompt variation into what is meant to be boot variation and inflated this figure to 29.92%. The pooled number is reported separately in the artifact and is not a boot-dispersion estimate.

Pooling every valid default-arm boot at concurrency 1 (n = 23):

24.8 28.4 29.5 31.9 32.1 33.2 37.0 39.1 40.6 41.1
41.3 43.7 44.2 44.5 45.5 47.3 50.7 60.8 61.2 62.0 64.7 66.5 73.6

We previously reported this as bimodal and retract that. An earlier draft claimed two discrete modes, on the basis of five boots with a gap in the middle. At n = 23 the gap is filled: fitting one versus two Gaussian components gives BIC 190.50 against 192.10, a ΔBIC of -1.60 *favouring a single component*. The retraction is recorded in the artifact. It is the same error we document elsewhere in this paper — reading structure off a sample too small to support it — and we made it in our own headline.

The dispersion is associated with the CUDA-graph path; we do not identify the mechanism. vLLM emits, at every speculative boot, a warning that `CUDAGraphMode.FULL_AND_PIECEWISE` is unsupported with spec-decode for the FlashInfer backend, and falls back. Pinning `cudagraph_mode: PIECEWISE` lands at the low end (30.04 versus a 30.57 default mean). Disabling CUDA graphs collapses the dispersion to CV 1.44% over 8 boots.

That is localisation, not causation, and we are explicit about the gap. `--enforce-eager` changes the entire execution path: graph capture, memory reservation, boot feasibility (§7) and FP8 kernel performance all move with it. We did not separate boot state from host state — rented L4 instances differ in physical GPU, clock and power behaviour between runs, and we did not record GPU identity. The defensible claim is that *the dispersion appeared in graph-enabled DFlash runs and collapsed under eager execution, and which graph, scheduler, allocator, kernel or host property is responsible remains unknown.*

The sharpest instance. One content-addressed cell (`dflash | bf16 | fp8_e4m3 | FLASHINFER`, graphs on) was measured in two ordinary grid runs:

Session	measurements (tok/s)	mean	τ
grid 1	28.59, 29.39, 33.81	30.60	4.183
grid 2	64.89, 71.77	68.33	4.211
ratio		**2.23**×	0.68% apart

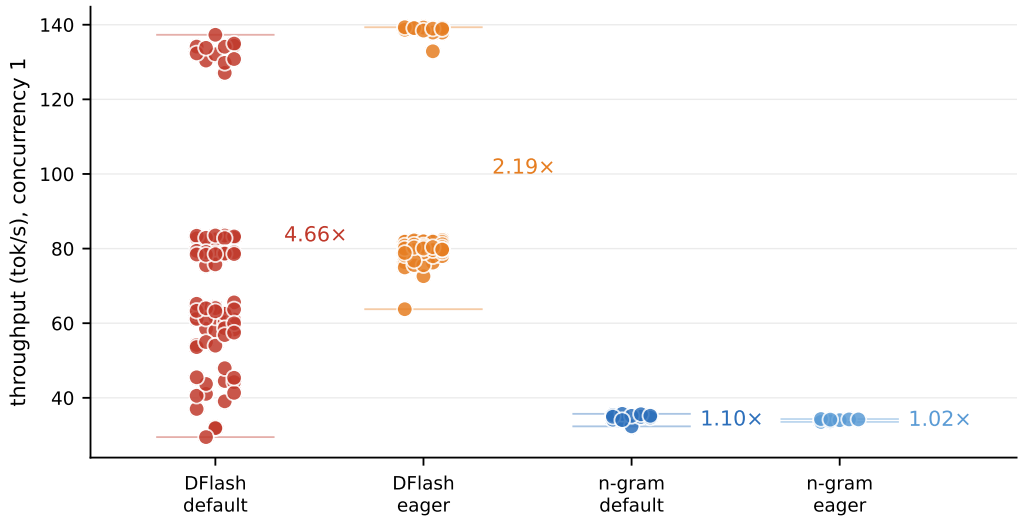


Figure 1: One dot per boot at concurrency 1, *fixed-prompt boots only* so that only the boot varies. DFlash's default arm is a wide, continuously filled distribution (CV 13.92%, $n = 12$); its eager arm and both n-gram arms are tight (1.44%, 2.67%, 0.85%). Every eager boot exceeds every default boot. *This figure plots the 2026-09-23 session only.* A second session of the identical configuration gives CV 3.73% with mean 61.2 tok/s, so this distribution's width is not reproducible; see §4. Two earlier versions of this figure were wrong in opposite directions: one showed five boots and was captioned as two clusters, the other pooled grid runs whose prompts differ and so plotted prompt variation alongside boot variation.

Same `cell_id`, therefore the same configuration by construction, and a matching `argv` fingerprint. The τ difference of 0.68% is within its measured dispersion (§5). Every earlier demonstration of this effect compared *something* — different arms, different cells, boots inside a test built to show it. This compares nothing.

We report no speedup ratio from this effect. Measuring both arms inside a single run, with the default boot landing high in its range, gives 1.13 × /1.06 × /1.05× at concurrency 1/16/64. Against a low-end boot the same eager numbers give 2.19×. The ratio is a statement about where in the default arm's distribution the comparison boot happened to land, not about the flag.

What is stable, within a session, is the *ordering*: CV 13.92% over 12 default boots against 1.44% over 8 eager boots, and every eager boot exceeds every default boot. The default arm disperses more than the eager arm beside it in every session we have run (13.92 vs 1.44; 3.73 vs 1.69; 0.99 vs 1.28 on 0.30.0, where the ordering finally inverts). The dispersion *magnitude* is not stable and we no longer describe it as such. Turning CUDA graphs off does not reliably make speculative decoding faster by a stated factor — it makes it *reproducible*, and whatever throughput it gains is however far down the distribution the default would otherwise have fallen.

Our first write-up of this section led with 2.38×, obtained by comparing against an arm we assumed representative, which was one draw from the wide distribution this same section describes.

It may be specific to draft-model speculation; our evidence is weaker than we first reported. Running the identical protocol with n-gram (prompt-lookup) speculation gives CV

2.67% across twelve boots against DFlash's 13.92%, and no benefit from `--enforce-eager`. But those twelve n-gram boots pool two container sessions (CV 3.19% at $n = 4$ and 1.55% at $n = 8$), and neither is the session that produced DFlash's 13.92%. Since DFlash's own dispersion ranges from 3.73% to 13.92% across sessions, n-gram's 3.19% is not distinguishable from DFlash's quieter session. The comparison is cross-session and the localisation is suggestive, not established. These are the same fact arriving twice: if throughput is already reproducible, there is nothing for the flag to stabilise. As a measurement sanity check, n-gram's acceptance is deterministic ($\tau = 1.732$ on every boot, spread 1.000×) where DFlash's spans 4.0914–4.1479 over the same fixed-prompt boots, which is what a prompt-matching mechanism should do. (The wider 4.09–4.43 seen elsewhere in our records spans different prompt sets and is not comparable.)

Twelve boots cannot prove absence; they bound it. At matched $n = 12$, DFlash gives CV 13.92% and n-gram 2.67%, a factor of five apart. That argument holds only against DFlash's noisiest session: matched against its quieter one the two are 3.73% and 2.67%, and the factor of five disappears. We report this as a weak localisation and note that settling it requires n-gram and DFlash measured in the *same* session, which we have not done. The consequence is that a claim like ours has to name the mechanism: "speculative throughput is unreproducible" would be refuted by the first reader who tries prompt-lookup.

5 Confound 3: acceptance dispersion is small, and had not been measured

Across 12 boots of an identical configuration with identical prompts, acceptance has a coefficient of variation of **0.62%** (mean τ 4.1055, sd 0.0256, 95% CI on the mean [4.0893, 4.1218]).

Unlike the throughput dispersion, this one replicates. We later measured the same quantity in two further sessions, one on the same engine release and one on 0.30.0:

Engine	Session	n	Mean τ	CV
0.29.0	2026-09-23	12	4.1055	0.62%
0.29.0	2026-09-29	12	4.1255	0.30%
0.30.0	2026-09-29	12	4.1498	0.61%

Three sessions, two engine releases, at least two physical cards: mean τ within 1% and dispersion below 0.7% throughout. This is the contrast that makes §4 worth reporting. On the same boots, acceptance is reproducible and reproducibly quiet, while throughput dispersion ranges over a factor of fourteen. Whatever varies between boots is not changing which tokens the drafter gets accepted; it is changing how fast they are produced.

We previously called this a "noise floor" and withdraw that. An earlier revision reported the *range* of four boots, 1.32%, and described it as a floor below which effects were "unfalsifiable". That was wrong in three ways, and we state them because the error is instructive.

A range is not a dispersion statistic that can be compared or reused: it grows with sample size by construction. Measured again at $n = 12$, the same quantity's range is **1.38%** — it increased, which is what ranges do and what floors do not. Second, a detection threshold depends on variance, sample size, power and the chosen test, none of which a range encodes; effects below it are not unfalsifiable, they are undetected at that n. Third, we transplanted a figure measured

at $\tau \approx 4.1$ on GSM8K at 768 tokens onto an earlier result measured at $\tau \approx 2.71$ on a different prompt set at 256 tokens.

What the data supports, and what it does not. It is tempting to convert 0.62% into a detection threshold and declare the 0.7% FP8 precision effect "not detected". We decline, for a reason that is the whole point of this section.

The 12 replicated boots are DFlash, GSM8K, 768-token budget, at $\tau \approx 4.1$. The 0.7% precision effect comes from *single-boot* cells at $\tau \approx 2.71$ on a different prompt set at a 256-token budget. There are not twelve boots per precision arm; there is one. Using dispersion measured at one operating point as an uncertainty bound for a comparison made at another is precisely the transplant we criticise two paragraphs above, and an earlier revision of this paper did it anyway.

The defensible statement is therefore weaker and shorter: *our FP8 precision comparison is unreplicated, and cannot establish either a difference or an equivalence.* Establishing equivalence would need repeated boots per precision arm at a declared margin, which we did not run.

Acceptance is not continuously noisy. Across those 12 boots τ takes exactly **two distinct values**, 4.0914 (nine boots) and 4.1479 (three). Given fixed prompts, acceptance is near-deterministic with occasional discrete shifts, so modelling it as Gaussian measurement noise was the wrong frame independently of the sample-size error.

What survives. We withdraw a previously-published claim of ours that backend choice perturbs τ by 2.25%. That figure came from *one boot per backend*, and a single boot is not a basis for a timing claim on this stack (§4). It is withdrawn because it is unreplicated, which needs no appeal to a floor.

What we would claim, narrowly. We could not find a published dispersion estimate for τ under repeated identical boots, and effects smaller than the one we measure here have been reported as real elsewhere. The underlying practice, however, is not ours and we do not claim it. Two concurrent papers arrive at it independently, from two further directions. Kaplan [9] calibrates an LLM judge by scoring "two ordinary runs of a model given the same prompts" and adds a null condition "provably identical in distribution…whose measured difference must be zero". Pape et al. [10] reach the same prescription from cross-engine reproducibility, recommending that "researchers should avoid relying on single evaluation runs" and that results be averaged across multiple seeds. Three independent arrivals at one principle — from output quality, from backend reproducibility, and from serving throughput — is better evidence that the principle is right than any one of them alone, and we would rather say so than be found to have cited whichever we happened to read first. Our contribution is to apply it to acceptance, and to report that the dispersion is small rather than to assert a threshold.

Operating point matters. Our 0.62% is measured at $\tau \approx 4.1$, GSM8K, 768-token budget. The same configuration at a 256-token budget on a different prompt set gave $\tau \approx 2.71$ — a 55% difference from generation length and prompt distribution alone. A τ quoted without both is close to uninterpretable, and the literature quotes τ freely.

6 Confound 4: "inter-token latency" is inter-chunk latency

The first three confounds add error. This one reverses the ranking.

A streaming client measures the gap between *arrivals*. vLLM emits one SSE chunk per decoding step, and a speculative verification step emits every accepted draft token at once. So a metric computed from stream-arrival timestamps — which is how our own harness computed it, and the natural reading of a streaming API — is *inter-chunk* latency, and on a speculative server it exceeds the amortised production time per emitted token by roughly τ.

We make no claim about which specific load generators do this. We did not audit any, and saying "every load generator" without naming and versioning them would be the kind of unsupported universal this paper objects to. What we establish is the mechanism and that one harness — ours — fell into it.

Table 2: Concurrency 64, CUDA graphs on, FP8 weights, auto KV, identical prompts and SLO.

	measured gap	tokens/chunk	amortised per token	throughput
speculative (DFlash)	**93.8 ms**	**4.122**	**22.8 ms**	**2219 tok/s**
non-speculative	33.8 ms	1.007	33.6 ms	1281 tok/s

The 22.8 ms figure is **amortised production time per emitted token**, not a per-token latency: those four tokens become visible to the client together, at one instant, 93.8 ms apart. Speculation produces tokens 1.5× faster on that amortised basis and appears 2.8× slower on the recorded metric. The `tokens_per_chunk` of 4.122 tracks the measured τ of $\approx$4.2, as it must.

Table 3: Goodput at p95 ITL $\leq$ 50 ms, TTFT $\leq$ 1000 ms, concurrency 64.

Configuration	raw req/s	SLO met	goodput req/s
dflash | fp8 | auto	**8.31**	**0%**	**0.000**
dflash | bf16 | auto	6.13	0%	0.000
none | fp8 | auto	5.20	100%	**5.188**
none | bf16 | auto	3.37	6%	0.209

Under this objective every speculative configuration receives zero compliant goodput while producing the most tokens per second in the grid.

This is not an artifact of the 50 ms threshold. Sweeping the ITL target at TTFT $\leq$ 1000 ms:

conc	mech	20 ms	35 ms	50 ms	75 ms
1	dflash	0%	64%	100%	100%
1	none	0%	89%	99%	100%
16	dflash	0%	0%	74%	99%
16	none	0%	89%	98%	100%
64	**dflash**	**0%**	**0%**	**0%**	**0%**
64	none	0%	3%	99%	100%

At concurrency 64 the speculative configuration scores zero at every threshold we tested, while the non-speculative one reaches 99% at 50 ms. The mechanism is not a coincidence of threshold

choice: the chunk cadence at that concurrency is 93.8 ms, so *any* inter-arrival target below the cadence fails by construction, and any target above it would pass trivially. At concurrency 1 and 16 the effect does not bite.

What this does and does not show. It shows that an SLO computed from stream-arrival gaps — which is how load generators compute ITL — inverts the ranking under speculation at the concurrency we tested. It does *not* show that all reasonable service objectives do so. We measured one workload, one arrival pattern (closed-loop fixed concurrency rather than open-loop arrival rates), and four thresholds. An objective defined over token-delivery curves, or over TTFT and end-to-end latency, would behave differently, and we did not measure chunk-size distributions.

This is not an engine defect or a bug in our harness: a server cannot deliver four tokens at four separate times when it produced them at one time. A client measuring arrival gaps will see this wherever a decoding step emits more than one token. At concurrency 1 the effect is present but does not bite — gaps are small in absolute terms and speculative cells meet the same target 100% of the time. It appears exactly where serving operates.

We do not claim the quotient is the right metric. Dividing by `tokens_per_chunk` recovers an average and discards the burstiness an ITL target exists to capture; whether a reader perceives four tokens every 94 ms as smoother than one every 34 ms is a question we did not study. Our claim is narrower: *ITL computed from stream arrivals does not measure what its name says under speculation*, and on this stack, at this concurrency, using it to gate a deployment inverts the decision. Whether a given deployment's tooling computes it that way is a question for that tooling. Prior work (§9) reports single-request latency, where the effect does not bite — consistent with it going unreported.

7 Measurements after diagnosing four confounds

We say *after diagnosing* rather than *with all four controlled*. Three are controlled: the attention backend is pinned, the execution path is carried as an axis rather than fixed, and timing is reported as chunk cadence. The fourth is not — draft-side backend selection remains uncontrolled (§3), because vLLM offers no way to pin it.

7.1 The execution-path flag is not a free control

We ran the full grid twice, 16 boots each, in independent containers, with `enforce_eager` as an *axis* rather than a pinned setting. Ratios are graphs-off over graphs-on, so below 1.0 means eager is slower.

`--enforce-eager` costs FP8-weight configurations a fifth to a half of their throughput and costs BF16 essentially nothing. Individual boots do not overlap on any FP8 row and overlap on every BF16 row, in both grids. τ is unmoved throughout.

The direction replicates on every cell; the coefficient does not, and that is itself a result. The graphs-on arm reproduces everywhere (0.95–1.06×), the eager arm reproduces for BF16 (1.00–1.05×) and does not for FP8 weights (1.22–1.40×). The eager+FP8 arm is boot-unstable — tight within a boot (20.2 and 20.5 tok/s on grid 1's two repeats) and 1.4× between them.

This study therefore found *two* unstable dimensions, each with a stable partner: speculation is unreproducible with CUDA graphs on and stable with them off; FP8 weights are stable with

Table 4: Effect of `--enforce-eager`, two independent grids.

Configuration	conc	grid 1	grid 2
none | bf16 | auto	1	0.95×	0.96×
none | bf16 | auto	64	0.96×	0.96×
none | **fp8** | auto	1	**0.49**×	**0.68**×
none | **fp8** | auto	64	**0.66**×	**0.78**×
none | **fp8** | fp8_e4m3	1	**0.50**×	**0.66**×
dflash | bf16 | auto	1	1.13×	1.32×
dflash | **fp8** | auto	1	**0.61**×	**0.76**×

graphs on and unstable with them off. Neither is visible in a single boot, and neither predicts the other.

A third constraint on the same flag: it cannot boot speculative cells with FP8 KV at all on this card, failing on an 892 MiB float32 logits buffer whose size is fixed by scheduler width times speculative width times vocabulary. Had we pinned eager — the obvious response to §4 — every FP8-weight baseline would have been reduced, every speculative FP8-KV cell would have failed to boot, and the speculation-versus-baseline speedups would have been inflated on exactly the cells this paper is about, with nothing in the output to indicate it.

7.2 A pre-registered prediction, and how it did

Before running the second grid we recorded a prediction: that the apparent ≈3× FP8-weight speculative advantage was mostly mode assignment, that it would survive but shrink to roughly 1.2–1.4×, and that FP8 rows would move least between arms.

	grid 1	grid 2
bf16 speculative, c=1, graphs off	79.5 tok/s	83.6 tok/s
FP8-weight advantage, graphs **on**	**1.69**×	**1.90**×
FP8-weight advantage, graphs **off**	**0.91**×	**1.09**×

Part 1 holds in both grids. Part 3 holds, but for the wrong reason: FP8 rows move most in absolute terms, downward, because eager penalises them. Part 2 fails: the ≈3× does shrink, but to 1.7–1.9× rather than the predicted 1.2–1.4×, and the further collapse to parity under graphs-off is a *second* effect that did not exist in our model when the prediction was written. Two effects, one of them unknown at the time, landing near a band guessed for the other. One of three held cleanly, and we report it as measured.

7.3 KV capacity

With the backend held fixed, FP8 KV cache approximately doubles token capacity, as arithmetic predicts.

The FLASHINFER row is not noise, and the anomaly is speculation-specific. Every other pinned-backend row doubles to within 0.3%; `dflash | bf16 | FLASHINFER` reaches only 1.789×. Comparing the two backends that serve *both* precisions, at each precision separately:

For the non-speculative configuration the spread is unchanged by precision, which is what halving the bytes per token should do. For the speculative one it grows to 1.118×. So the divergence is

Table 5: KV-cache token capacity, BF16 versus `fp8_e4m3`. Rows marked † use `auto` backend selection, where the backend differs between columns (§3) and the ratio is therefore not attributable to precision alone.

Mech	Weights	Backend	BF16 KV	FP8 KV	ratio
none	bf16	FLASHINFER	73,888	147,776	2.000×
none	bf16	TRITON_ATTN	77,168	153,872	1.994×
none	fp8	FLASHINFER	102,208	204,384	2.000×
dflash	bf16	FLASHINFER	64,784	115,920	1.789×
dflash	bf16	TRITON_ATTN	64,912	129,584	1.996×
dflash	fp8	FLASHINFER	77,616	155,232	2.000×
none	bf16	`auto`†	77,168	147,776	1.915×
dflash	bf16	`auto`†	58,080	115,920	1.996×

Configuration	KV precision	between-backend spread	difference
dflash	BF16	1.002×	128 tokens
dflash	FP8	**1.118×**	**13,664 tokens**
none	BF16	1.044×	3,280 tokens
none	FP8	1.041×	6,096 tokens

not a property of FP8 KV in general — it appears when a draft model is also allocating KV.

We report the observation and not a mechanism: we did not profile the allocator, and block-size rounding, workspace reservation and per-model layout are all consistent with it. It is noted because it compounds §3. The backend `auto` selects under FP8 KV is FlashInfer, which is also the one realising least of the theoretical doubling on speculative configurations, so a practitioner who switches KV dtype and takes the default gets less capacity than the arithmetic promises — and gets it silently.

7.4 Task accuracy

Over 256 GSM8K problems with a 768-token budget, scored from stored generations, quantizing weights, the KV cache, or both produces no detectable change in accuracy.

Table 6: Accuracy at $n = 256$, backend pinned to FLASHINFER. No cell produced an unparseable generation.

Mech	Weights	KV cache	Accuracy	95% CI
none	bf16	auto	85.9%	81.1–89.7
none	bf16	fp8_e4m3	87.5%	82.9–91.0
none	fp8	auto	88.3%	83.8–91.7
none	fp8	fp8_e4m3	87.1%	82.4–90.7
dflash	bf16	auto	86.3%	81.6–90.0
dflash	bf16	fp8_e4m3	87.9%	83.3–91.3
dflash	fp8	auto	86.3%	81.6–90.0
dflash	fp8	fp8_e4m3	87.1%	82.4–90.7

Across all twelve within-mechanism paired comparisons (McNemar, exact binomial), discordant pairs number 16–23 of ≈250, p ranges from 0.238 to 1.000, and every 95% interval on the dif-

ference is contained within ±4.7 percentage points. This bounds the effect rather than demonstrating its absence: at $n = 256$ a degradation of two or three points would not be detected.

This is a task-level null, not a numerical one. Low-precision kernels in vLLM are known to disagree with each other numerically: Chen [11] finds two interchangeable INT8 GEMM implementations producing different outputs on every one of 64 sequences on SM89, with FP8 divergence growing with reduction depth where INT8 remains at parts-per-million. Nothing above contradicts that, and nothing above tests it. We compare task accuracy end to end at $n = 256$; a difference in logits large enough to flip tokens can still leave a GSM8K score untouched, and our intervals are wide enough to hide several points. Read this table as "no accuracy cost we could detect at this sample size", not as evidence that FP8 arithmetic is numerically equivalent.

8 Threats to validity

- **This is a single-configuration case study.** Every result is one NVIDIA L4 (SM89), one target model (Qwen3-4B), one learned draft checkpoint, one engine release (vLLM 0.29.0), one principal workload (GSM8K), and one decoding configuration. Backend selection and CUDA-graph behaviour all depend on engine version, kernels, GPU architecture and model, so none of these findings should be assumed to transfer. What transfers is the *class* of measurement error, not the magnitudes.
- **Boot and host effects are not separated.** Runs used rented instances; we did not record GPU serial, clocks or power state, so a host-to-host component cannot be distinguished from a boot-to-boot one in §4.
- **Configuration identity is not byte identity.** Our `cell_id` hashes the declared configuration; model revisions are declared as `main`, a mutable ref, and dependencies as open ranges. We resolved the commits post hoc and their upstream modification dates precede every run, but the harness did not record them at launch. See `docs/reproducibility.md`.
- **The mechanism is unidentified.** We show that the dispersion is present with CUDA graphs enabled and collapses under eager execution. We do not show which graph, scheduler, allocator, kernel or host property is responsible, and we did not record GPU identity, so a host-to-host component is not excluded.
- **Mechanisms are not established** for the FP8-KV boot failure or the FP8-weight eager penalty. Both are observed and reproduced; neither was profiled. We state the readings we find plausible and mark them as readings.
- **Client-side saturation is argued, not measured.** A single-process benchmarking client can itself queue under load and have the delay attributed to the server [12]. Our concurrency is 64 and our generations are hundreds of milliseconds long, which places us one to two orders of magnitude below the ∼1000 QPS onset those authors report, and §6's argument is structural rather than timing-precision dependent. We did not, however, run a client-saturation control — for instance against a zero-latency mock server — so this rests on a load-regime argument rather than on a measurement of our own harness.
- **Which claims are replicated, and which are single-session.** §4 argues that a quantity measured once is a draw, so we state where this paper meets that standard. *Replicated:* the throughput dispersion and the default/eager ratio (five sessions, three

cards, two engine releases); acceptance dispersion (three sessions); the eager penalty on FP8 weights, whose direction reproduces across two independent grids while its coefficient does not, which is why we report a range. *Structural rather than statistical, and so not subject to this:* backend selection under dtype change and the ITL/chunk identity, both categorical facts read from engine logs. *Single-session, and to be read as draws:* the A10 dispersion, the 0.30.0 dispersion at each individual card, and the goodput figures at concurrency 64.

- **Grid records carry no session identity.** `probes.jsonl` records a `probe_run` per container; `sweep.jsonl` records nothing equivalent. So for any result derived from the grid — capacity, accuracy, goodput — the question §4 shows to matter, *how many sessions is this*, cannot be answered from the records. We found this by trying to ask it. The fix is one field and it is not retrospective.
- **The draft-side backend confound is acknowledged, not removed.**
- **Our retraction list covers errors we caught.** The same blindness that produced them applies to finding them. We assume undiscovered instances exist; one was found the same day we wrote the list.
- **No base rate.** Comparable studies do not report their retractions, so our count is evidence of legibility, not of unusual error-proneness.

Every finding in the artifact carries a mandatory "what this does not establish" section, and this paper inherits those limits rather than softening them.

9 Related work

None reports boot-level dispersion, and the single-request results in this table are vulnerable to the confound in §4.

On benchmarks for this exact question. SPEED-Bench [5] is the closest thing to a standard for evaluating speculative decoding, integrates with vLLM, and sweeps concurrency. It reports that synthetic inputs overestimate real-world throughput and that optimal draft length depends on batch size. It does not report dispersion across repeated identical runs, which is the axis §4 concerns, and a benchmark that does not is one more place the error term in §4 would go unnoticed.

On the accuracy axis of the same switch. Pape et al. [10] run Qwen3-4B — our target model — across five inference engines on H100 and L40, the latter sharing our SM89 architecture, with twelve seeds per configuration. They report that backend choice shifts GSM8K accuracy by up to 16.6 percentage points, and that *disabling CUDA graphs* shifts accuracy "by up to +0.15%". They measure no throughput or latency at all.

That pairing is worth stating plainly. The execution-path switch that moves output agreement by 0.15% is the same switch across which we measure throughput dispersion of 3.73–13.92% depending on the session (§4), on the same model family and the same GPU architecture. We do not read this as a contradiction: they are two different quantities, and a switch may reasonably leave one alone while moving the other. We read it as the clearest available evidence that the throughput axis of this switch has not been examined — a careful, multi-seed, cross-engine reproducibility study probed CUDA graphs and reported a null, because the quantity it measured was not the one that moves.

Chen [11] supplies the complementary observation at the kernel level, also on SM89: two nominally interchangeable INT8 GEMM kernels in vLLM agree on no sequence in 64, while each arm reproduces itself *bit-for-bit across cold restarts*. Byte-identical output across boots and unreproducible throughput across boots are therefore both properties of this stack, and a study that establishes the first says nothing about the second. We note that Chen reports FP8 GEMM divergence growing with reduction depth where INT8 stays at parts-per-million; that is element-level disagreement between implementations, and our §7 accuracy null is an end-to-end task measurement, so the two do not speak to each other directly.

On client-side measurement artifacts. Chandrasekar and Kramberger [12] show that a single-process benchmarking client becomes the bottleneck under load, adding client-side queuing delay that is then attributed to the server; they validate this against a zero-latency simulator, so the artifact is provably in the client. Their distortion sets in around 1000 QPS, where the clients they test saturate at 146–443 QPS. Our measurements run at concurrency 64 with generation latencies in the hundreds of milliseconds, one to two orders of magnitude below that onset, so we do not believe this mechanism affects our numbers — but it is the same *kind* of error as §6, in which the client's view of the token stream, rather than the server, produces the misleading metric. They do not discuss speculative decoding or multi-token chunk arrival, which is the specific case §6 concerns.

On "goodput". TurboSpec [6] optimises a quantity of that name in vLLM, so the word needs care. Theirs is the rate of *successfully generated (accepted) tokens*, used as a control signal; it involves no latency target. Ours is the rate of requests *meeting a latency service objective*. The two are different measurements sharing a name, and §6 concerns only the latter.

Two relationships deserve stating precisely rather than by omission.

Liu et al. [7] report that acceptance varies "within a request, across different requests, and across different datasets". That is variation of the *quantity*; §5 measures the *instrument* — how much a repeated identical measurement moves. The two are complementary, and conflating them would be a mistake in either direction. They also establish that increasing batch size systematically reduces SD's relative speedup, which is prior art for load-dependence and we claim no novelty there.

Kong et al. [8] validate a latency model on vLLM SD measurements to $R^2 = 0.997$ using a single-sweep methodology with no repeated runs and no discussion of CUDA graphs. Our §4 identifies an error term that such a protocol cannot see. We do not claim their results are wrong — they measure A100/H100 where we measure L4, and §4 localises the effect to draft-model speculation — only that the term is unmeasured and worth checking.

On hardware. Liu et al. use H100; Kong et al. use A100 and H100; both use FP16 KV. The regime where KV capacity actually binds, and where FP8 KV therefore has something to buy, is the smaller card. That is why we measure on L4, and it is the gap this paper occupies.

10 Artifact

Harness, raw records, derived tables and findings: https://github.com/akshathtiwari/spec-fp8-study.

Archived, with a DOI that resolves to the current version: 10.5281/zenodo.23033168. Each release is archived separately, so the version of this paper that made a claim remains retrievable after the claim is restated — which has already happened once: v1.0.0 reported the §4 dispersion as a single figure, and v1.1.0 replaced it with the two measurements and the ratio reported above.

Two scripts are worth naming. `analysis/check_provenance.py` verifies that every finding's

citations resolve to stored records, that stored ids still recompute from their stored configs, and that no result rests on terminal output; it exits with the defect count. `analysis/check_paper.py` verifies that every numeric figure in this paper appears in the underlying records. Both run in `paper/build.sh` before LaTeX does, so a paper that fails its own audit is not typeset, and both have found real defects in this document — including two tables that had silently diverged from the findings they summarise.

`check_provenance.py` reports two defects, and we state them rather than suppress them: findings F009 and F014 rest on probe output predating the harness's per-measurement recording, so their numbers are not independently checkable from the repository. Neither is cited in this paper. The remaining twenty-seven findings resolve to stored records. `paper/build.sh` stops the build on any *additional, undisclosed* defect; the two above are an explicit allowance, so the gate is against regression rather than against a nonzero count.

What content-addressing does and does not prove. A `cell_id` guarantees two records came from the same *declared* configuration. It does not guarantee the same model bytes, engine build, or dependency versions. `argv_fingerprint` and the per-record `env` block close part of the gap; `docs/reproducibility.md` states the rest, including the resolved model commits and the five changes that would make the artifact airtight.

The entire study was run on rented per-second L4 time at a cost in the tens of dollars. The error analysis in §3–§6 is what that budget can establish, and on the evidence of §9 it is what the field is missing — rather than another single-run speedup number measured on hardware most readers cannot rent.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. *ICML*, 2023. arXiv:2211.17192.

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318, 2023.

[3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.

[4] K. Cobbe et al. Training verifiers to solve math word problems. arXiv:2110.14168, 2021.

[5] T. Abramovich, M. Ashkenazi, I. Putterman, B. Chislett, T. Mitra, B. Darvish Rouhani, R. Zilberstein, and Y. Geifman. SPEED-Bench: a unified and diverse benchmark for speculative decoding. arXiv:2604.09557, 2026.

[6] X. Liu, J. Park, L. Hu, W. Kwon, Z. Li, C. Zhang, K. Du, X. Mo, K. You, A. Cheung, Z. Deng, I. Stoica, and H. Zhang. TurboSpec: closed-loop speculation control system for optimizing LLM serving goodput. arXiv:2406.14066, 2024–2025.

[7] X. Liu, J. Yu, J. Park, I. Stoica, and A. Cheung. Speculative decoding: performance or illusion? *MLSys*, 2026. arXiv:2601.11580.

[8] L. Kong, M. Flynn, M. Peng, N. Shavit, M. Kurtz, and A. Marques. An interpretable latency model for speculative decoding in LLM serving. arXiv:2605.15051, 2026.

[9] J. Kaplan. A calibrated instrument for measuring how inference optimizations affect output quality. arXiv:2609.18005, 2026.

[10] D. Pape, J. Evertz, and L. Schönherr. The silent hyperparameter: quantifying the impact of inference backends on LLM reproducibility. arXiv:2605.19537, 2026.

[11] T.-R. Chen. The integer alibi: localizing cross-kernel divergence in INT8-quantized LLM inference. arXiv:2608.13756, 2026.

[12] A. Chandrasekar and J. Kramberger. Identifying and mitigating systemic measurement bias in production LLM inference benchmarks. arXiv:2605.24217, 2026.

[13] Speculative decoding meets quantization: compatibility evaluation and hierarchical framework design. arXiv:2505.22179.

[14] SpecKV: speculative decoding under KV-cache compression. arXiv:2605.02888.

[15] QSpec: speculative decoding with complementary quantization schemes. *EMNLP*, 2025. arXiv:2410.11305.

[16] QuantSpec: self-speculative decoding with hierarchical quantized KV cache. arXiv:2502.10424.

[17] ML-SpecQD: multi-level speculative decoding with quantized drafts. arXiv:2503.13565.

[18] Quasar: quantization-aware speculative decoding. arXiv:2603.01399.

Work	What it did	Gap this addresses
Performance or Illusion? [7]	Systematic SD evaluation, vLLM on H100, CUDA graphs enabled, FP16 KV	No run-to-run variance; no FP8 KV; no SLO goodput; ITL not decomposed
Interpretable latency model [8]	Models SD latency vs request rate, vLLM on A100/H100, R^2 to 0.997	Single-sweep methodology; no repeats, error bars, or CUDA-graph discussion
Calibrated instrument [9]	Calibrates an LLM judge via a null condition; measures its per-sample noise	Same principle, applied to output quality rather than throughput or acceptance
SPEED-Bench [5]	Benchmark standardising SD evaluation, vLLM and TensorRT-LLM, throughput split across concurrencies	No run-to-run dispersion; no ITL/chunk distinction; no SLO goodput
Silent hyperparameter [10]	Backend choice shifts GSM8K accuracy by 16.6 pp across 5 engines; Qwen3-4B on H100 and L40 (SM89); 12 seeds per configuration	Measures output agreement only; no throughput or latency, so the CUDA-graph axis is probed for accuracy and not for dispersion
Integer alibi [11]	Two nominally interchangeable INT8 GEMM kernels in vLLM agree on no sequence; RTX 4090 (SM89)	Kernel-level output divergence, not serving throughput; prefill regime only
Measurement bias [12]	Client-side GIL queuing distorts latency metrics; onset at $\sim$1000 QPS	Client saturation, not engine behaviour; no speculative decoding or chunked streaming
TurboSpec [6]	Closed-loop speculation control in vLLM, optimising "goodput"	Goodput there is accepted-token rate, not SLO attainment; no latency objective
Spec. Decoding Meets Quantization [13]	EAGLE-2 with W8A8, W4A16, W4A8; single-request	No FP8, no KV-cache quantization, not batched
SpecKV [14]	Adaptive γ under compression	Algorithmic, step-level, not serving goodput
QSpec [15]	W4A4 draft + W4A16 verify, shared weights	Shared-weight special case, not FP8
QuantSpec [16]	Self-speculation, 4-bit hierarchical KV	A method, not a cross-mechanism measurement
ML-SpecQD [17], Quasar [18]	Quantized drafts, W8A8 verification	Neither reports availability or boot-level variance